

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Исследование уязвимостей веб-приложений и программных
интерфейсов, настройка механизмов аутентификации»
Тема «Идентификация задачи»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рулько М.А.

Проверил:

Хацкевич А.С.

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Задание 1. Обход аутентификации через SQL-инъекцию и ее устранение

POST

▼

http://127.0.0.1:5000/login

ParamsAuthorizationHeaders (9)BodyPre-request ScriptTestsSettings

☐ none

☐ form-data

☒ x-www-form-urlencoded

☐ raw

☐ binary

	Key	Value
☒	username	' OR '1'='1
☒	password	' OR '1'='1

Иза ввода в credentials итоговый SQL query будет таким :

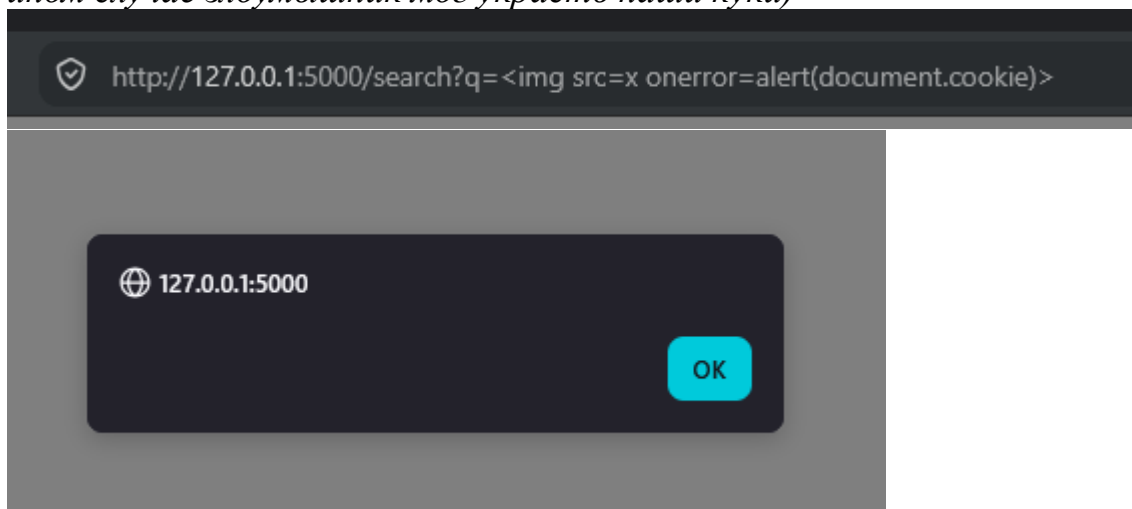
```
SELECT id, username, role FROM users WHERE username="" OR '1'='1' AND  
password="" OR '1'='1'
```

И вернет нам JWT токен

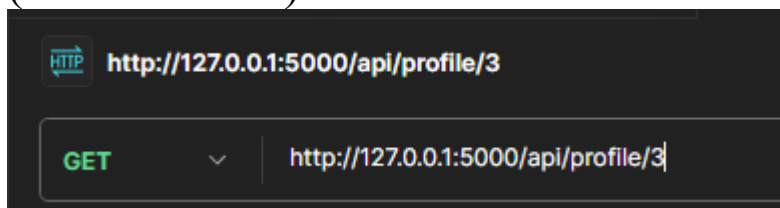
Body			Cookies (1)	Headers (6)	Test Results
Name	Value	Domain			
auth_token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozL0J1c2VybmFtZSI6ImFkbWlulwIiwicm9sZSI6ImFkbWluliwiZXh0IjoxNzkwMTUwODc0fQ.sUGyCGq4yymm54TGgv-eEEr1Gr2j7-RW3iKctmz3L48	127.0.0.1			

Задание 2. Эксплуатация межсайтового скриптинга (XSS)

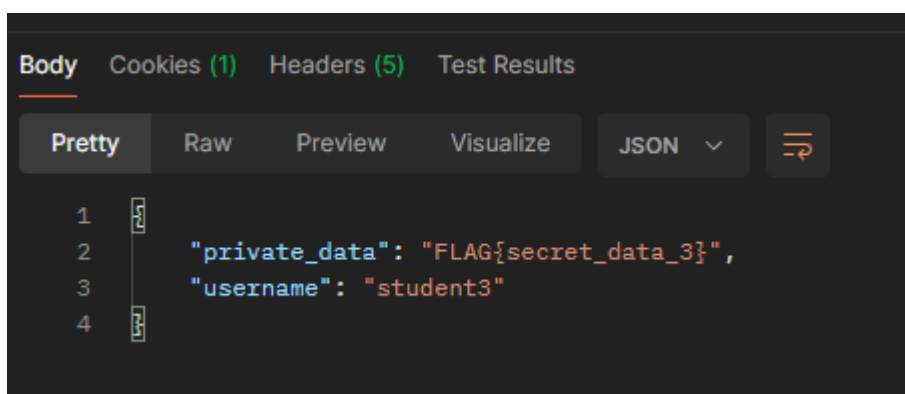
Отсутствие экранирования пользовательского ввода перед вставкой в HTML, из-за чего текст превращается в исполняемый код в браузере жертвы. (*alert для простоты – в ином случае злоумышник мог украсть наши куки*)



Задание 3. Нарушение разграничения доступа на уровне объектов (BOLA/IDOR)



Утечка — доступ к чужим данным по прямому ID без проверки владельца сессии.



Задание 4. Подделка и атака на JWT-токен

```
decoded = self.decode_complete(
[+] Секрет найден: student2026
{'user_id': 1, 'username': 'admin', 'role': 'admin', 'exp': 1790110878}
(venv) PS F:\SMZKS-2026-autumn>
```

```
2 <p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>
```

```
# исправление 1: SQL-инъекция устранена — запрос параметризован,
# пользовательский ввод передаётся отдельно от текста запроса
# и не может изменить его структуру

@app.route('/login', methods=['POST'])
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
```

```

# исправление 2 (часть 2): добавляем заголовок Content-Security-Policy
# ко всем ответам сервера — script-src разрешает выполнение только
# скриптам с совпадающим nonce, инлайн-обработчики (onerror/onload и т.п.)
# выполняться не смогут в принципе
@app.after_request
def add_csp_header(response):
    response.headers['Content-Security-Policy'] = f"script-src 'nonce-{g.csp_nonce}'"
    return response

```

```

# исправление 3: BOLA/IDOR устранён — добавлена проверка,
# что запрашиваемый ID профиля совпадает с ID владельца текущей сессии
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')
    if not token:
        return jsonify({"error": "Unauthorized"}), 401
    try:
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
    except jwt.ExpiredSignatureError:
        return jsonify({"error": "Token expired"}), 401
    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401

    if decoded.get('user_id') != user_id:
        return jsonify({"error": "Forbidden"}), 403

```

```

# исправление 4: секрет больше не хардкодится коротким словом,
# а берётся из переменной окружения или генерируется криптостойко (256 бит)
JWT_SECRET = os.environ.get("JWT_SECRET", secrets.token_hex(32))

```

Вывод:

Разобрался с базовыми способами фрода веб приложения.